🚀 TECHLIFT – STAGE 2

Beginner Coding Challenge

Language: Python
Duration: 60 Minutes
Total Problems: 5
Total Marks: 100
Team Size: 2–3 members

Stage 2 Evaluation

Criterion	Weightage
Correctness	40%
Logic / Problem-Solving	25%
Code Quality	15%
Time Efficiency	10%
Successful Submission	10%
Total	**100%**

These weightages are from the official TECHLIFT event documentation.

🟢 QUESTION 1 — SUM OF DIGITS

Difficulty

Easy

Marks

15

Problem Statement

Given a positive integer N, write a Python program to calculate the **sum of all digits** of the number.

Input Format

The first line contains an integer N.

Output Format

Print the sum of the digits of N.

Constraints

$1 \leq N \leq 10^9$

Example 1

Input

12345

Output

15

Example 2

Input

908

Output

17

Explanation

For 12345:

1 + 2 + 3 + 4 + 5 = 15

🔑 Organizer Solution

```
n = int(input())

total = 0

while n > 0:
    digit = n % 10
    total += digit
    n //= 10
```

print(total)

How the code works

For:

12345

The program repeatedly extracts the last digit:

12345 % 10 → 5

1234 % 10 → 4

123 % 10 → 3

12 % 10 → 2

1 % 10 → 1

Then:

5 + 4 + 3 + 2 + 1 = 15

Test Cases

Input	Expected Output
12345	15
908	17
1000	1
7	7
456789	39

🟢 QUESTION 2 — COUNT EVEN AND ODD NUMBERS

Difficulty

Easy

Marks

15

Problem Statement

Given N integers, write a Python program to count:

1. Number of even numbers
2. Number of odd numbers

Input Format

The first line contains an integer N.

The second line contains N integers.

Output Format

Print:

Even: X

Odd: Y

where X is the number of even numbers and Y is the number of odd numbers.

Constraints

$1 \leq N \leq 100$

Each number will be between:

-1000 and 1000

Example

Input

6

10 15 22 7 8 13

Output

Even: 3

Odd: 3

Explanation

Even numbers:

10, 22, 8

Odd numbers:

15, 7, 13

Therefore:

Even = 3

Odd = 3

🔑 Organizer Solution

```
n = int(input())

numbers = list(map(int, input().split()))

even_count = 0
odd_count = 0

for num in numbers:
    if num % 2 == 0:
        even_count += 1
    else:
        odd_count += 1

print("Even:", even_count)
print("Odd:", odd_count)
```

Test Cases

Test Case 1

Input

6

10 15 22 7 8 13

Output

Even: 3

Odd: 3

Test Case 2

Input

5

2 4 6 8 10

Output

Even: 5

Odd: 0

Test Case 3

Input

5

1 3 5 7 9

Output

Even: 0

Odd: 5

Test Case 4

Input

4

-2 -3 6 7

Output

Even: 2

Odd: 2

🟡 QUESTION 3 — SECOND LARGEST NUMBER

Difficulty

Easy–Moderate

Marks

20

Problem Statement

Given a list of integers, find the **second largest distinct number**.

If the largest number appears multiple times, it should still be treated as one number.

Input Format

The first line contains an integer N.

The second line contains N integers.

Output Format

Print the second largest distinct number.

Constraints

$2 \leq N \leq 100$

The input will contain **at least two distinct numbers**.

Example

Input

6

10 25 8 25 18 5

Output

18

Explanation

The distinct numbers are:

5, 8, 10, 18, 25

Largest:

25

Second largest:

18

Therefore the answer is:

18

🔑 **Organizer Solution**

```
n = int(input())

numbers = list(map(int, input().split()))

unique_numbers = list(set(numbers))
unique_numbers.sort(reverse=True)

print(unique_numbers[1])
```

Important judging point

The word **distinct** is important.

For:

10 25 8 25 18 5

The answer is:

18

NOT:

25

Test Cases

Test Case 1

Input:

6

10 25 8 25 18 5

Output:

18

Test Case 2

5

10 20 30 40 50

Output:

40

Test Case 3

6

5 5 5 8 8 10

Output:

8

Test Case 4

5

100 50 75 25 90

Output:

90

Test Case 5

4

-10 -5 -20 -3

Output:

-5

🟡 QUESTION 4 — PALINDROME CHECKER

Difficulty

Moderate

Marks

20

Problem Statement

Given a string, determine whether it is a **palindrome**.

A palindrome is a word or sequence that reads the same from left to right and right to left.

Input Format

The input contains a single string.

Output Format

Print:

Palindrome

if the string is a palindrome.

Otherwise print:

Not Palindrome

Constraints

$1 \leq$ length of string ≤ 100

For this beginner challenge, assume the input contains a single word with no spaces.

Example 1

Input

madam

Output

Palindrome

Example 2

Input

hello

Output

Not Palindrome

Explanation

For:

madam

Reverse:

madam

Both are equal, so it is a palindrome.

For:

hello

Reverse:

olleh

They are different, so it is not a palindrome.

🔑 Organizer Solution

```
word = input().strip()


if word == word[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Test Cases

Input	Expected Output
madam	Palindrome
hello	Not Palindrome
level	Palindrome
python	Not Palindrome
radar	Palindrome
computer	Not Palindrome

🔴 QUESTION 5 — STUDENT SCORE ANALYZER

This is the **final and highest-mark problem**.

Difficulty

Moderate

Marks

30

Problem Statement

You are given the marks obtained by N students.

Write a Python program to calculate:

1. Average marks
2. Highest mark
3. Lowest mark
4. Number of students who scored **greater than or equal to the average**

Input Format

The first line contains an integer N.

The second line contains N integer marks.

Output Format

Print:

Average: X

Highest: Y

Lowest: Z

Above/Equal Average: W

Round the average to **2 decimal places**.

Constraints

$1 \leq N \leq 100$

$0 \leq \text{marks} \leq 100$

Example

Input

6

70 85 60 90 75 80

Calculation

Total:

70 + 85 + 60 + 90 + 75 + 80 = 460

Average:

460 / 6 = 76.666...

Rounded:

76.67

Highest:

90

Lowest:

60

Students scoring ≥ 76.67:

85, 90, 80

Count:

3

Output

Average: 76.67

Highest: 90

Lowest: 60

Above/Equal Average: 3

🔑 Organizer Solution

```
n = int(input())


marks = list(map(int, input().split()))
```

```
average = sum(marks) / n
highest = max(marks)
lowest = min(marks)

count = 0

for mark in marks:
    if mark >= average:
        count += 1

print(f"Average: {average:.2f}")
print(f"Highest: {highest}")
print(f"Lowest: {lowest}")
print(f"Above/Equal Average: {count}")
```

Test Cases

Test Case 1

Input

6

70 85 60 90 75 80

Output

Average: 76.67

Highest: 90

Lowest: 60

Above/Equal Average: 3

Test Case 2

Input

5

50 50 50 50 50

Output

Average: 50.00

Highest: 50

Lowest: 50

Above/Equal Average: 5

Test Case 3

Input

5

10 20 30 40 50

Output

Average: 30.00

Highest: 50

Lowest: 10

Above/Equal Average: 3

Test Case 4

Input

4

95 80 70 55

Output

Average: 75.00

Highest: 95

Lowest: 55

Above/Equal Average: 2

Test Case 5

Input

3

0 25 100

Output

Average: 41.67

Highest: 100

Lowest: 0

Above/Equal Average: 1

📊 FINAL STAGE 2 QUESTION SUMMARY

No.	Problem	Difficulty	Marks	Main Concepts
1	Sum of Digits	🟢 Easy	15	Loops, arithmetic
2	Count Even & Odd	🟢 Easy	15	Lists, conditions
3	Second Largest	🟡 Moderate	20	Lists, sorting, duplicates
4	Palindrome Checker	🟡 Moderate	20	Strings, slicing
5	Student Score Analyzer	🔴 Moderate	30	Lists, aggregation, conditions
	TOTAL		**100**	

🔐 Hidden Test Cases

For the actual competition, **do not give participants all the test cases above**.

Give them only the examples in the problem statement.

Keep additional cases hidden for evaluation.

For example:

Q1 hidden

1000000000 → 1

999999999 → 81

Q2 hidden

10

1 2 3 4 5 6 7 8 9 10

Expected:

Even: 5

Odd: 5

Q3 hidden

7

100 100 90 80 90 70 60

Expected:

90

Q4 hidden

racecar

Expected:

Palindrome

Q5 hidden

10

45 67 89 32 76 54 90 21 68 73

This prevents teams from simply hard-coding the sample outputs.

🏆 FINAL PARTICIPANT INSTRUCTIONS

Put these instructions at the top of the Stage 2 coding sheet:

TECHLIFT – STAGE 2: BEGINNER CODING CHALLENGE

1. The round duration is **60 minutes**.
2. There are **5 coding problems** carrying **100 marks**.
3. Use **Python** for all problems.
4. Read the input and output format carefully.
5. Programs must produce the required output exactly.

6. Test your programs with different inputs before submission.
7. Teams must submit their own solutions.
8. Sharing code between teams is not permitted.
9. Any use of unauthorized external assistance may lead to disqualification.
10. The decision of the evaluation team will be final.

The 2–3-member team structure is consistent with the event documentation.

🔥 **One important improvement**

For the **actual competition**, I would **not use only these five basic examples as the entire evaluation**. We should now create the **full Stage 2 competition package**:

Participant Question Paper → Hidden Test Cases → Python Answer Key → Evaluation Rubric → Team Score Sheet → Ranking/qualification rule for Stage 3.

That will make the round much more professional and prevent arguments during evaluation.